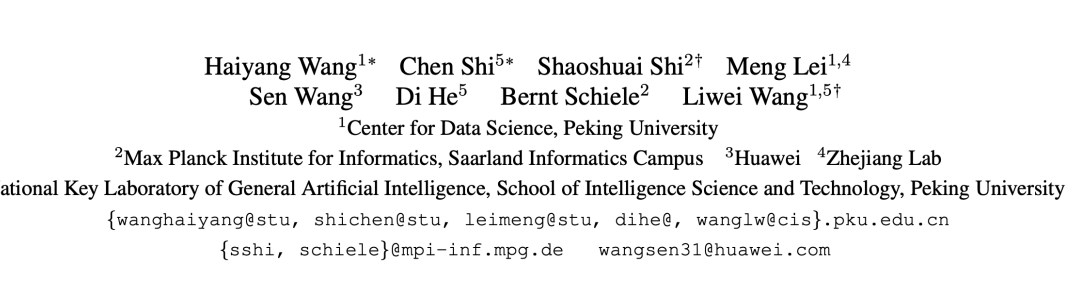
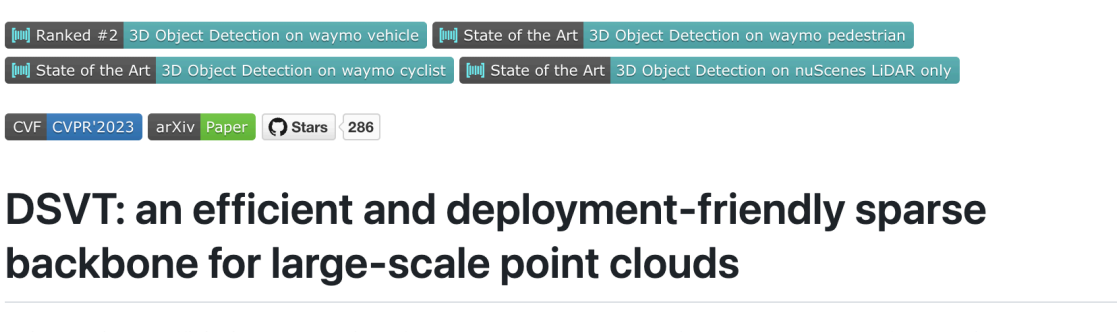
DSVT:

Dynamic Sparse Voxel Transformer with Rotated Sets



DSVT: Dynamic Sparse Voxel Transformer with Rotated Sets

Haiyang Wang[1*] Chen Shi[2*] Shaoshuai Shi[3†] Meng Lei[1,5]
Sen Wang[4] Di He[1] Bernt Schiele[3] Liwei Wang[1,2†]
[1]Center for Data Science, Peking University
[3]Max Planck Institute for Informatics, Saarland Informatics Campus [4]Huawei [5]Zhejiang Lab
[2]National Key Laboratory of General Artificial Intelligence, School of Intelligence Science and Technology, Peking University



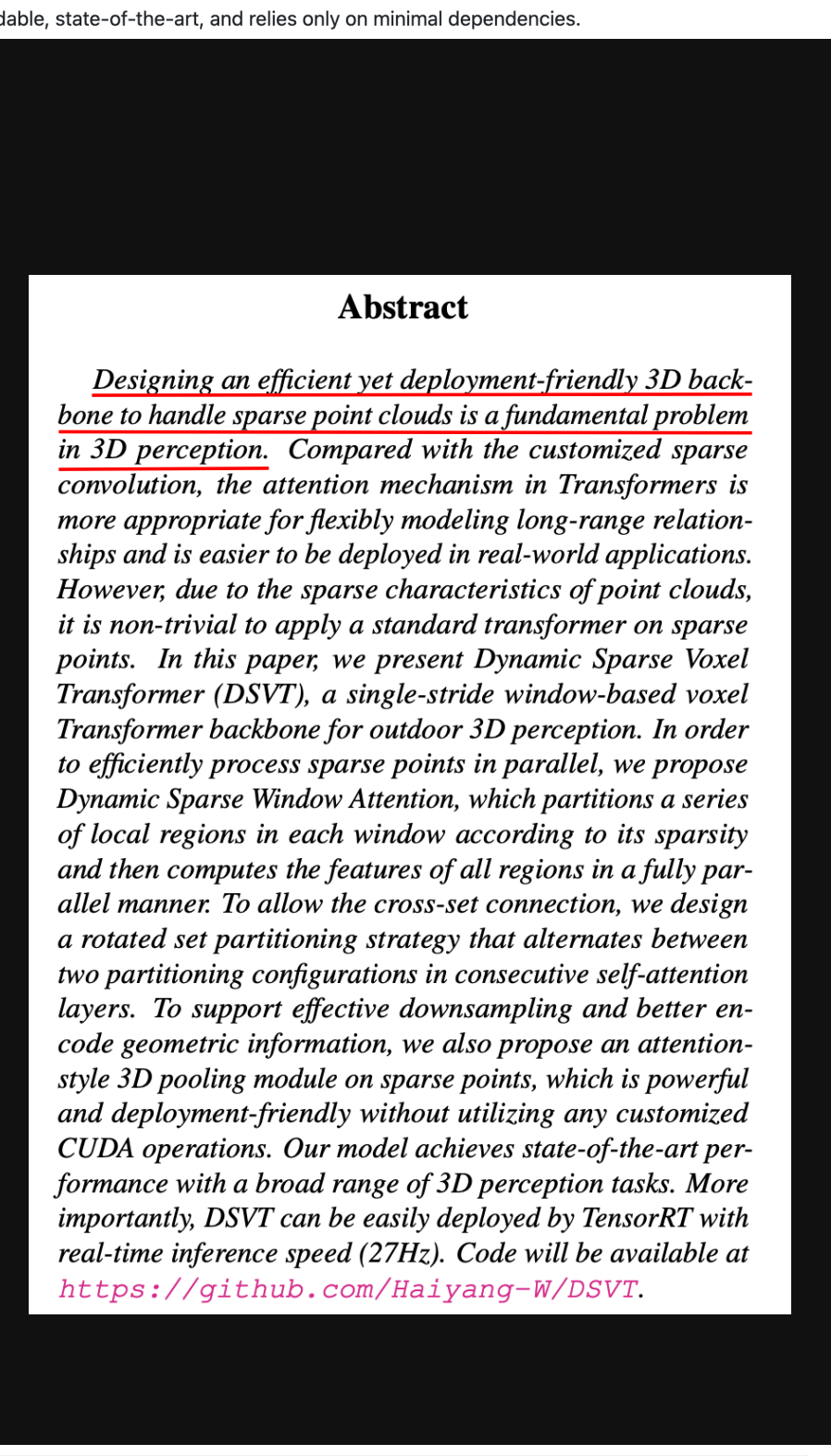
DSVT: an efficient and deployment-friendly sparse backbone for large-scale point clouds

This repo is the official implementation of CVPR paper: DSVT: Dynamic Sparse Voxel Transformer with Rotated Sets as well as the follow-ups. Our DSVT achieves state-of-the-art performance on large-scale Waymo Open Dataset with real-time inference speed (27Hz). We have made every effort to ensure that the codebase is clean, concise, easily readable, state-of-the-art, and relies only on minimal dependencies.

Abstract

"Designing an efficient yet deployment-friendly 3D backbone to handle sparse point clouds is a fundamental problem in 3D perception"

- What's sparse point clouds?
- Why is so important?

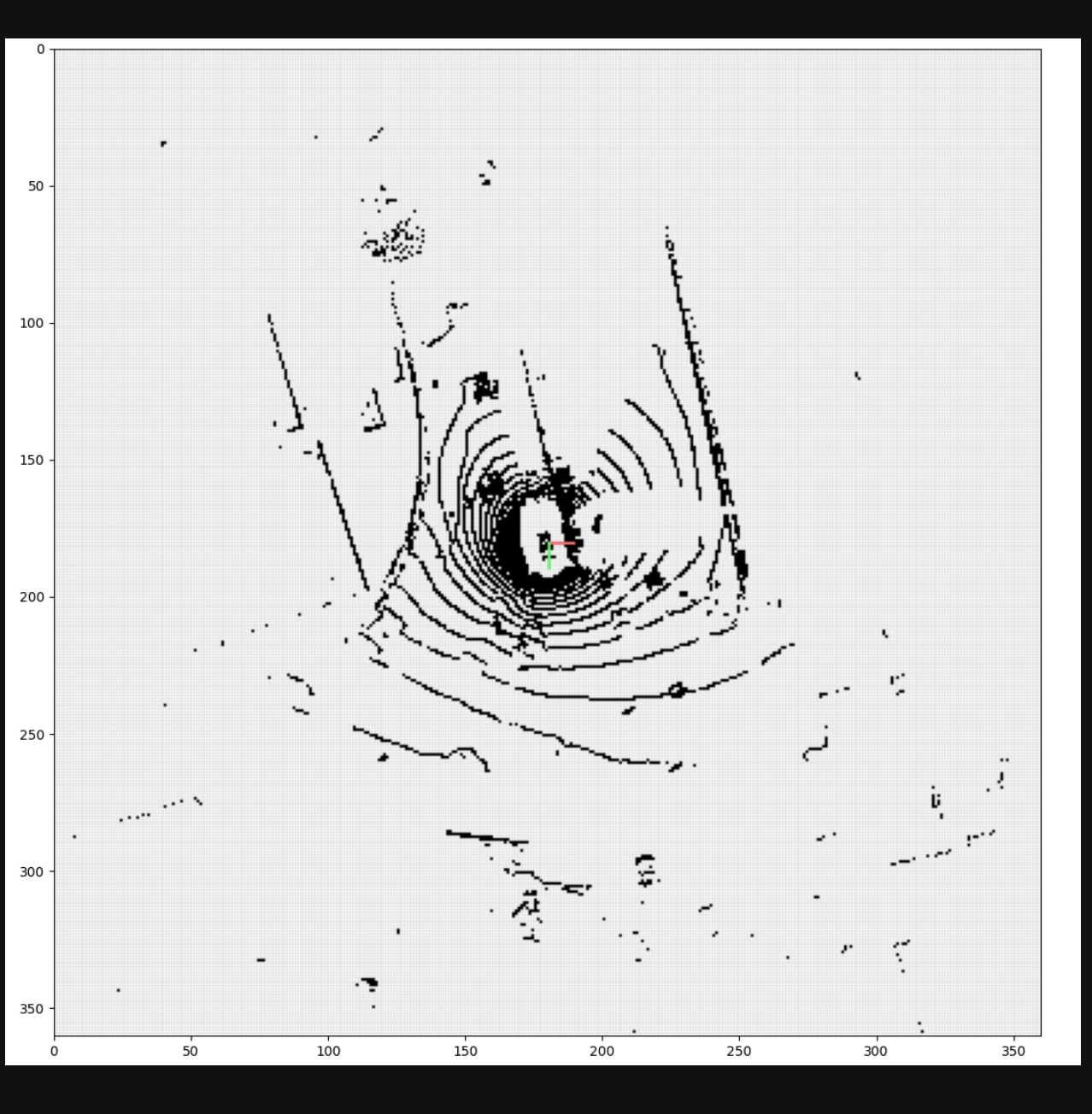


Abstract

Designing an efficient yet deployment-friendly 3D backbone to handle sparse point clouds is a fundamental problem in 3D perception. Compared with the customized sparse convolution, the attention mechanism in Transformers is more appropriate for flexibly modeling long-range relationships and is easier to be deployed in real-world applications. However, due to the sparse characteristics of point clouds, it is non-trivial to apply a standard transformer on sparse points. In this paper, we present Dynamic Sparse Voxel Transformer (DSVT), a single-stride window-based voxel Transformer backbone for outdoor 3D perception. In order to efficiently process sparse points in parallel, we propose Dynamic Sparse Window Attention, which partitions a series of local regions in each window according to its sparsity and then computes the features of all regions in a fully parallel manner. To allow the cross-set connection, we design a rotated set partitioning strategy that alternates between two partitioning configurations in consecutive self-attention layers. To support effective downsampling and better encode geometric information, we also propose an attention-style 3D pooling module on sparse points, which is powerful and deployment-friendly without utilizing any customized CUDA operations. Our model achieves state-of-the-art performance with a broad range of 3D perception tasks. More importantly, DSVT can be easily deployed by TensorRT with real-time inference speed (27Hz). Code will be available at https://github.com/Haiyang-W/DSVT.

Sparsity

- Point Clouds have move obvious sparsity
- sparse rate = empty pixel / image size
- 0.967 = 1 - 4200 / (360*360)



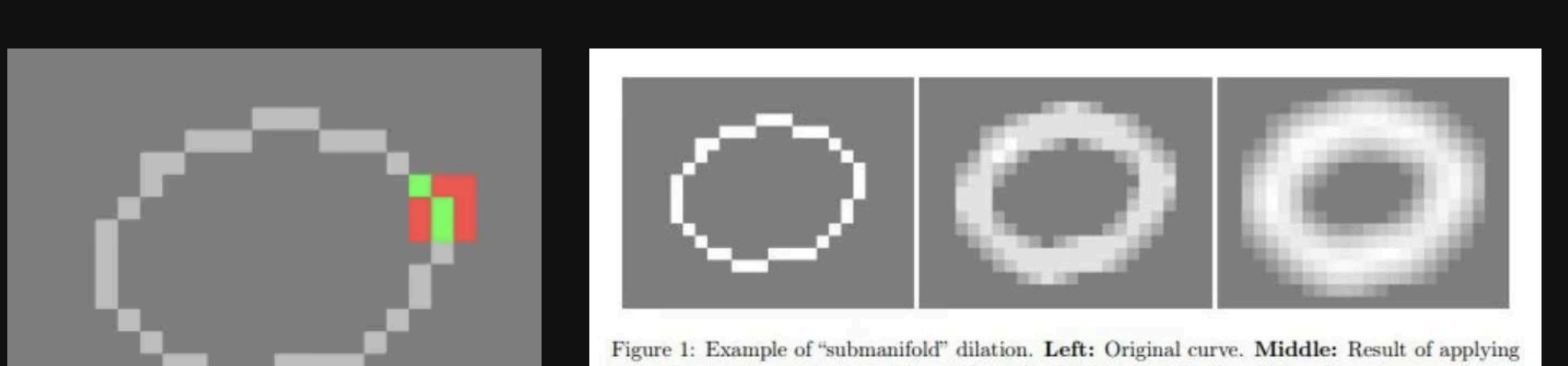
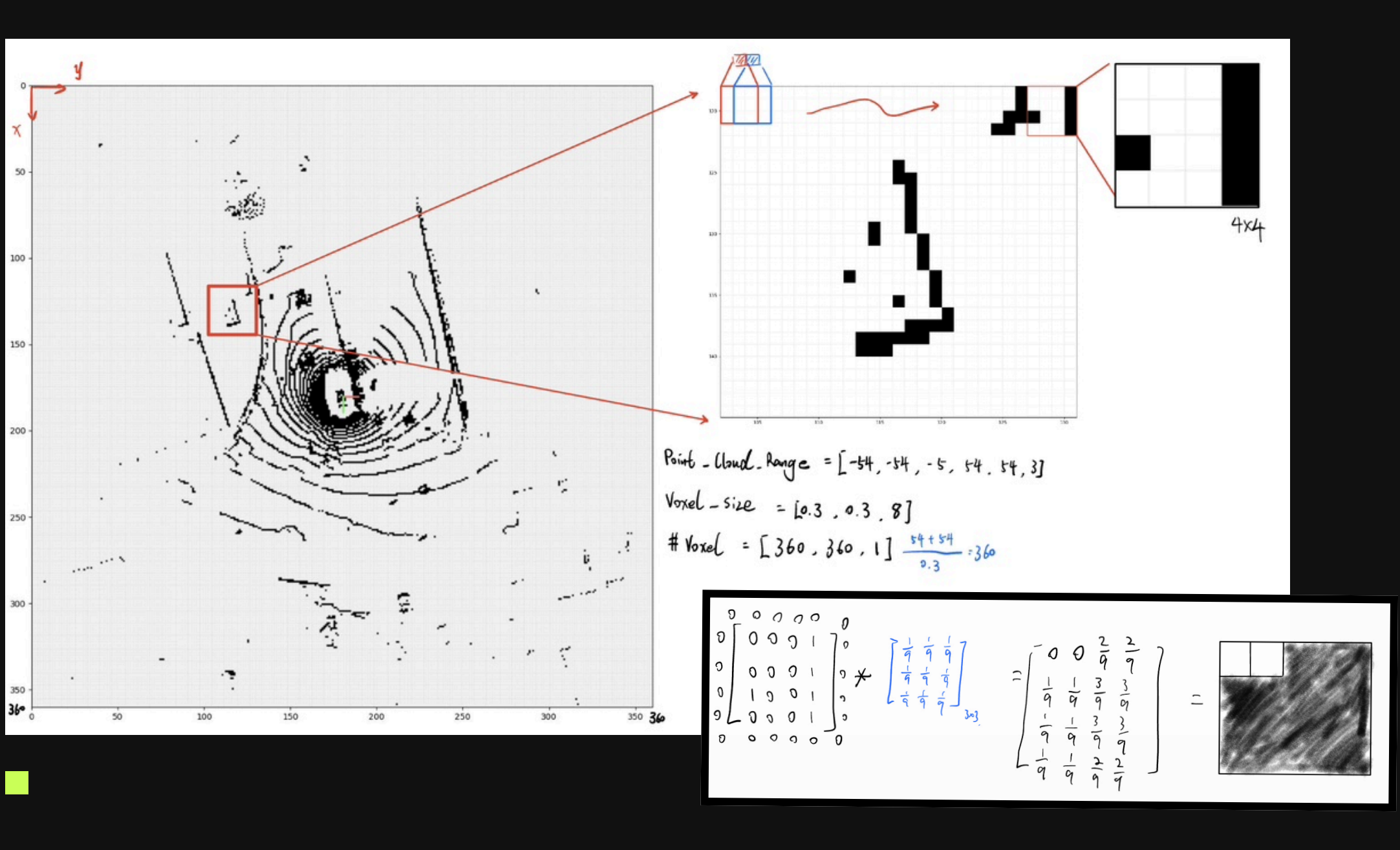


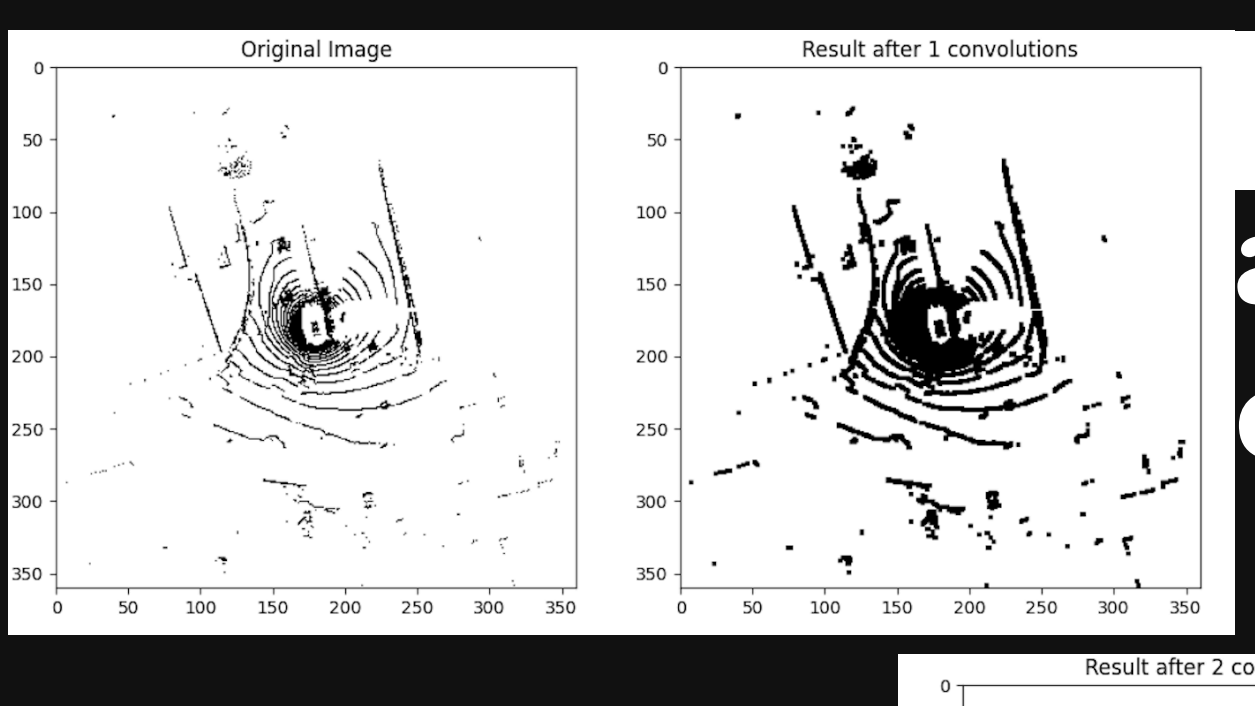
Figure 1: Example of "submanifold" dilation. Left: Original curve. Middle: Result of applying a regular 3 × 3 convolution with weights 1/9. Right: Result of applying the same convolution again. The example shows that regular convolutions substantially reduce the sparsity of the feature maps.

Apply Convolution to sparse data

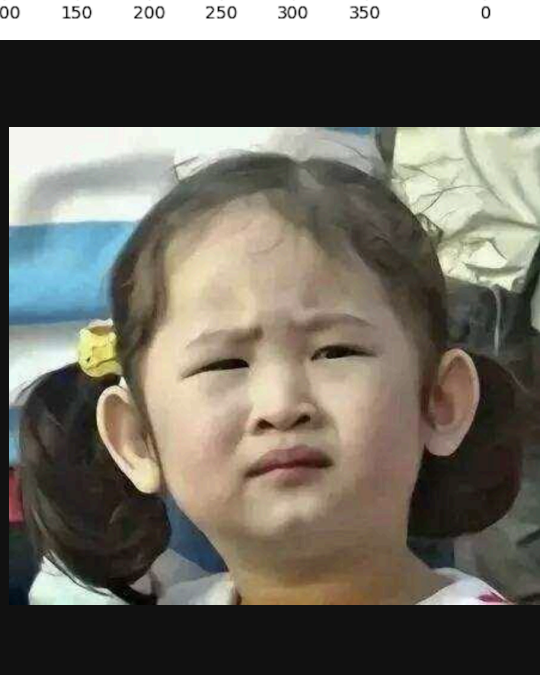
- Features diluted
- Structure blurred

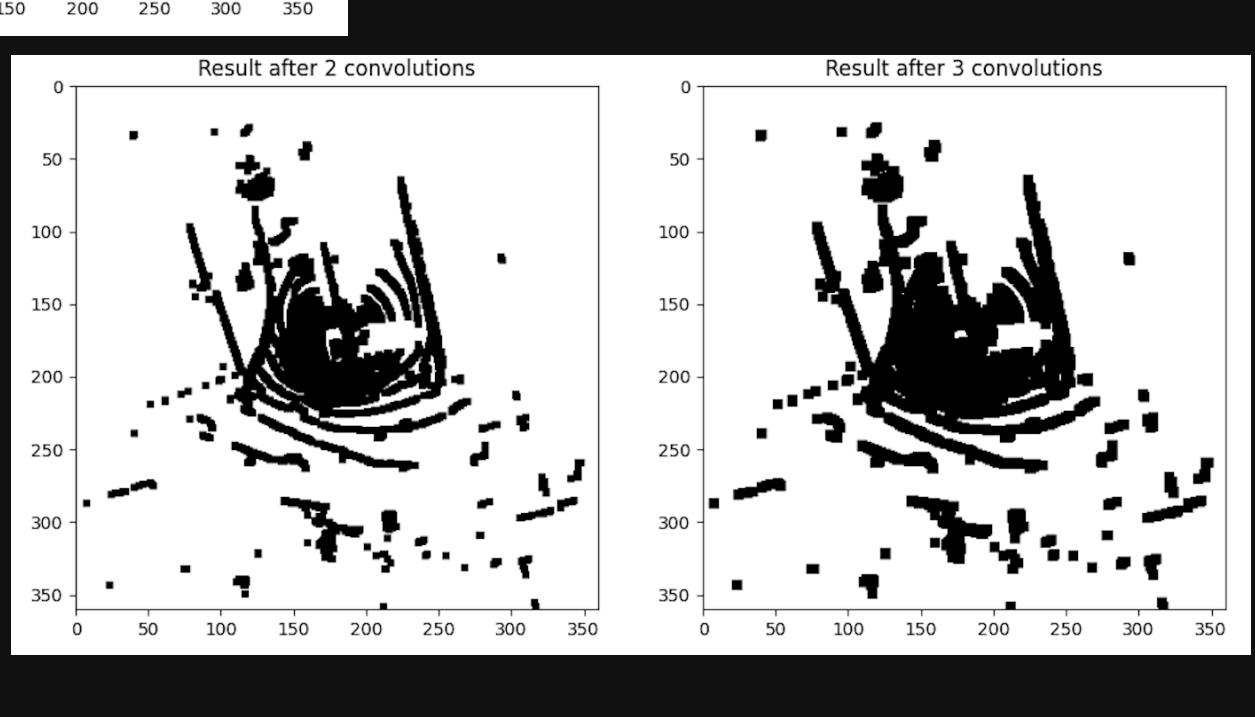
> "The 3x3 kernel passes the non-empty pixel to all empty area 😣





what happen after 4 times convolution?





What's sparse point clouds?

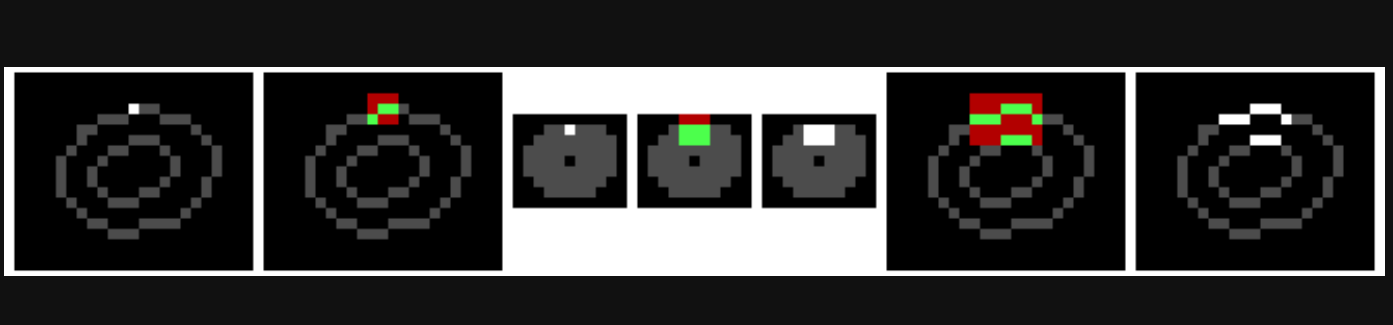
- Sparse rate = Empty pixel / Image size

Why is so important?

- The convolution kernel passes the non-empty pixel to all empty area

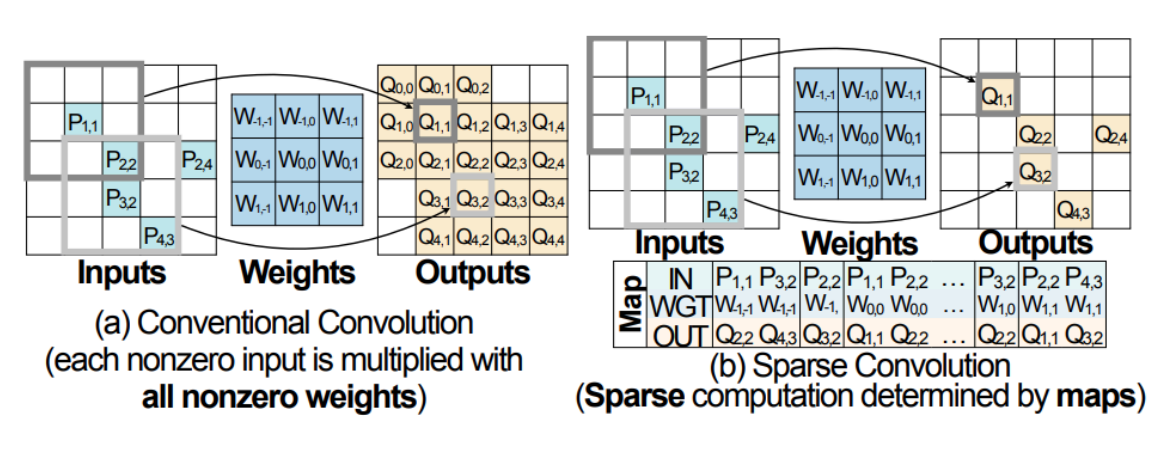
How to fix ?

- Sparse Convolution



Sparse Conv

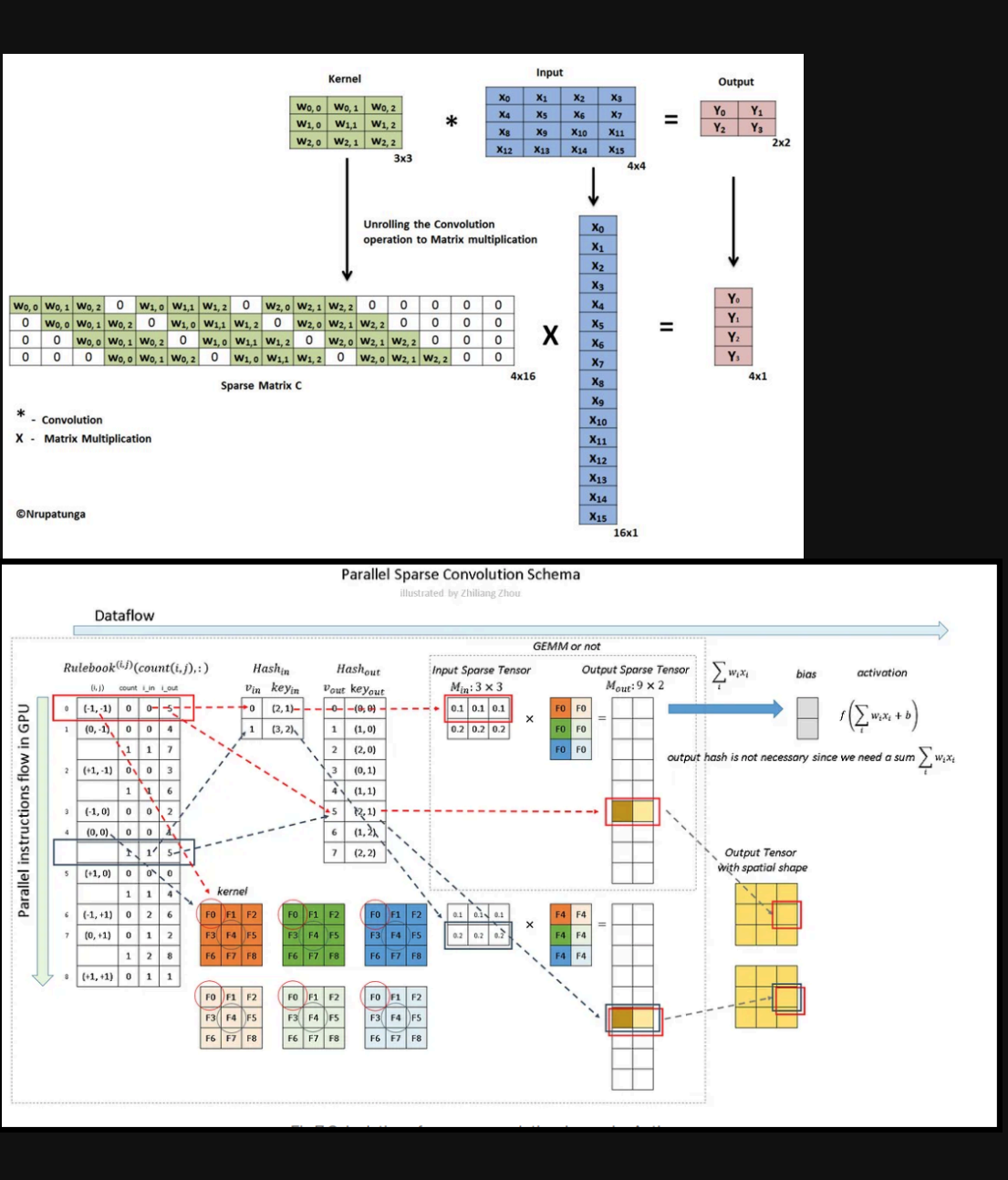
1. sparseconvnet
2. spconv
3. torchsparse
4. minkowskiengine



Inputs
Weights
Outputs
(a) Conventional Convolution
(each nonzero input is multiplied with all nonzero weights)
Inputs
Weights
Outputs
Map
IN
WGT
OUT
(b) Sparse Convolution
(Sparse computation determined by maps)

Figure 3: Sparse convolution (b) does *not* multiply each nonzero input with all nonzero weights as conventional convolution (a) does.

But, sparse sonvolution is so hard to deploy

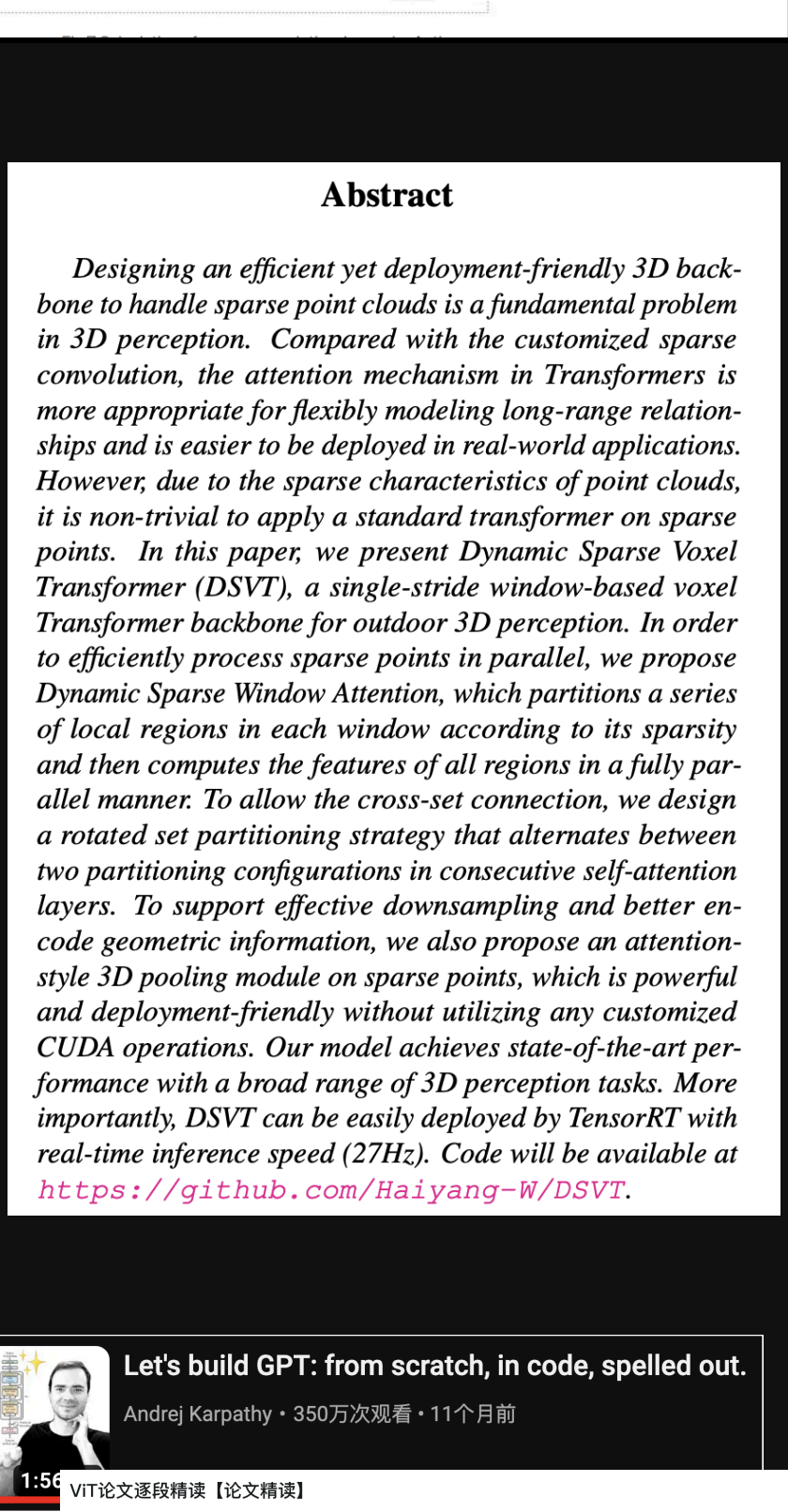


Kernel
Input
Output
Unrolling the Convolution operation to Matrix multiplication
Sparse Matrix C
Parallel Sparse Convolution Schema
Dataflow
Parallel instructions flow in GPU
Output Tensor with spatial shape

Abstract

"the attention mechanism in Transformer is more appropriate"

"a single-stride window-based voxel transformer backbone for outdoor 3D perception"

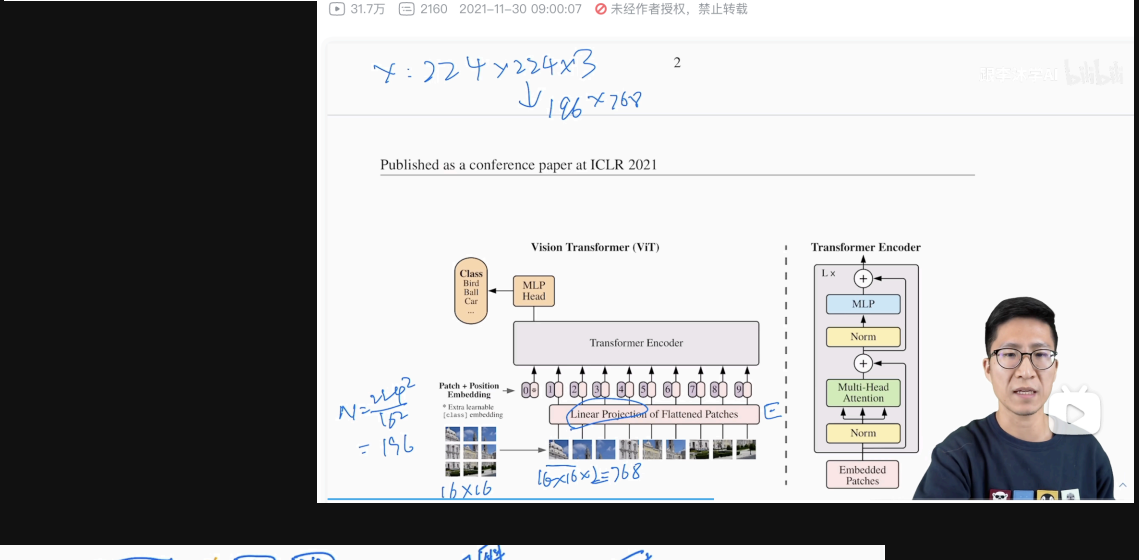


Abstract

Designing an efficient yet deployment-friendly 3D backbone to handle sparse point clouds is a fundamental problem in 3D perception. Compared with the customized sparse convolution, the attention mechanism in Transformers is more appropriate for flexibly modeling long-range relationships and is easier to be deployed in real-world applications. However, due to the sparse characteristics of point clouds, it is non-trivial to apply a standard transformer on sparse points. In this paper, we present Dynamic Sparse Voxel Transformer (DSVT), a single-stride window-based voxel Transformer backbone for outdoor 3D perception. In order to efficiently process sparse points in parallel, we propose Dynamic Sparse Window Attention, which partitions a series of local regions in each window according to its sparsity and then computes the features of all regions in a fully parallel manner. To allow the cross-set connection, we design a rotated set partitioning strategy that alternates between two partitioning configurations in consecutive self-attention layers. To support effective downsampling and better encode geometric information, we also propose an attention-style 3D pooling module on sparse points, which is powerful and deployment-friendly without utilizing any customized CUDA operations. Our model achieves state-of-the-art performance with a broad range of 3D perception tasks. More importantly, DSVT can be easily deployed by TensorRT with real-time inference speed (27Hz). Code will be available at https://github.com/Haiyang-W/DSVT.

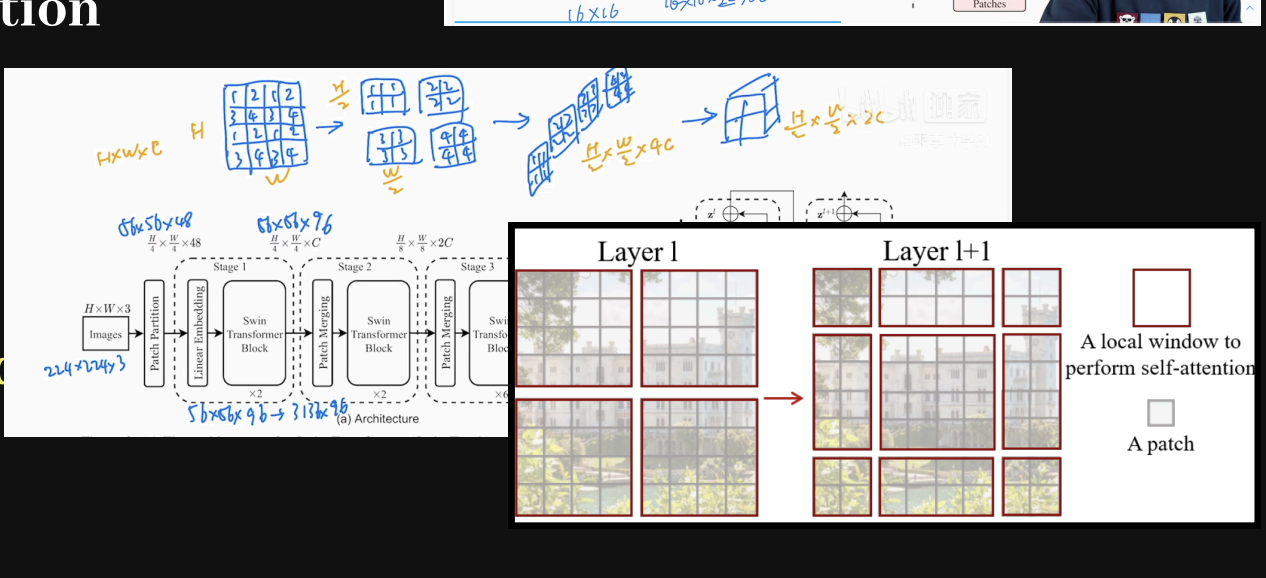
Transformer

- Attention is all you need (encoder / decoder)
- An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (Vit)
- Swin Transformer: Hierarchical Vision Transformer using Shifted Windows (Swin)

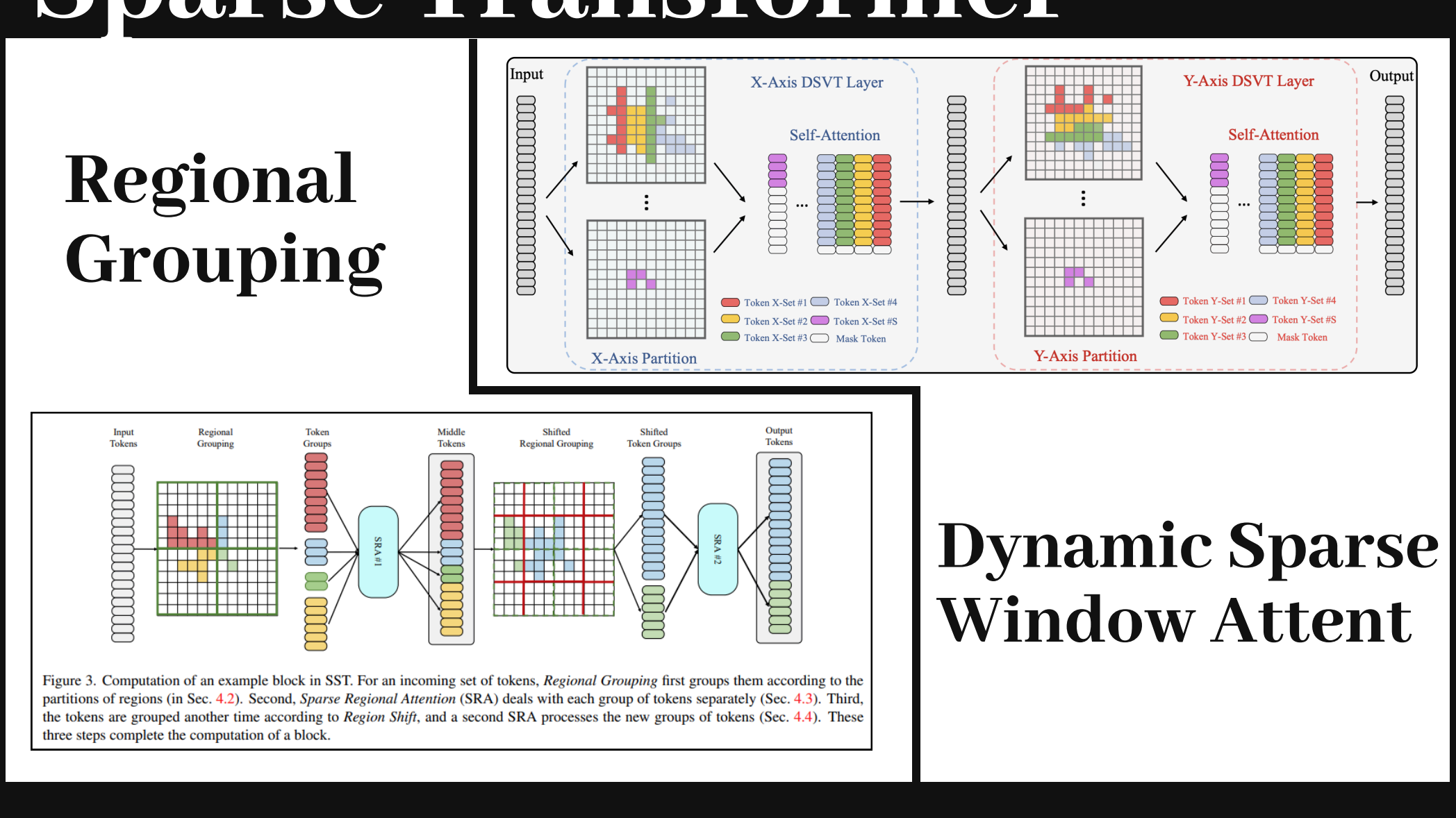


Let's build GPT: from scratch, in code, spelled out.
Andrej Karpathy・350万次观看・11个月前

ViT论文逐段精读【论文精读】



Sparse Transformer



Regional Grouping

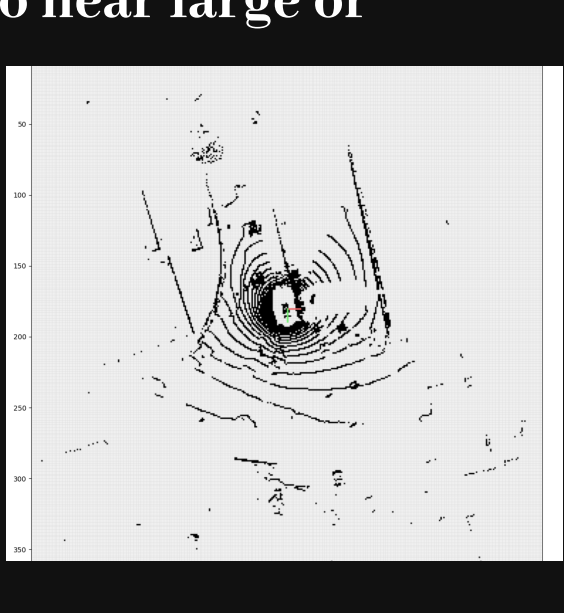
Figure 3. Computation of an example block in SST. For an incoming set of tokens, *Regional Grouping* first groups them according to the partitions of regions (in Sec. 4.2). Second, *Sparse Regional Attention* (SRA) deals with each group of tokens separately (Sec. 4.3). Third, the tokens are grouped another time according to *Region Shift*, and a second SRA processes the new groups of tokens (Sec. 4.4). These three steps complete the computation of a block.

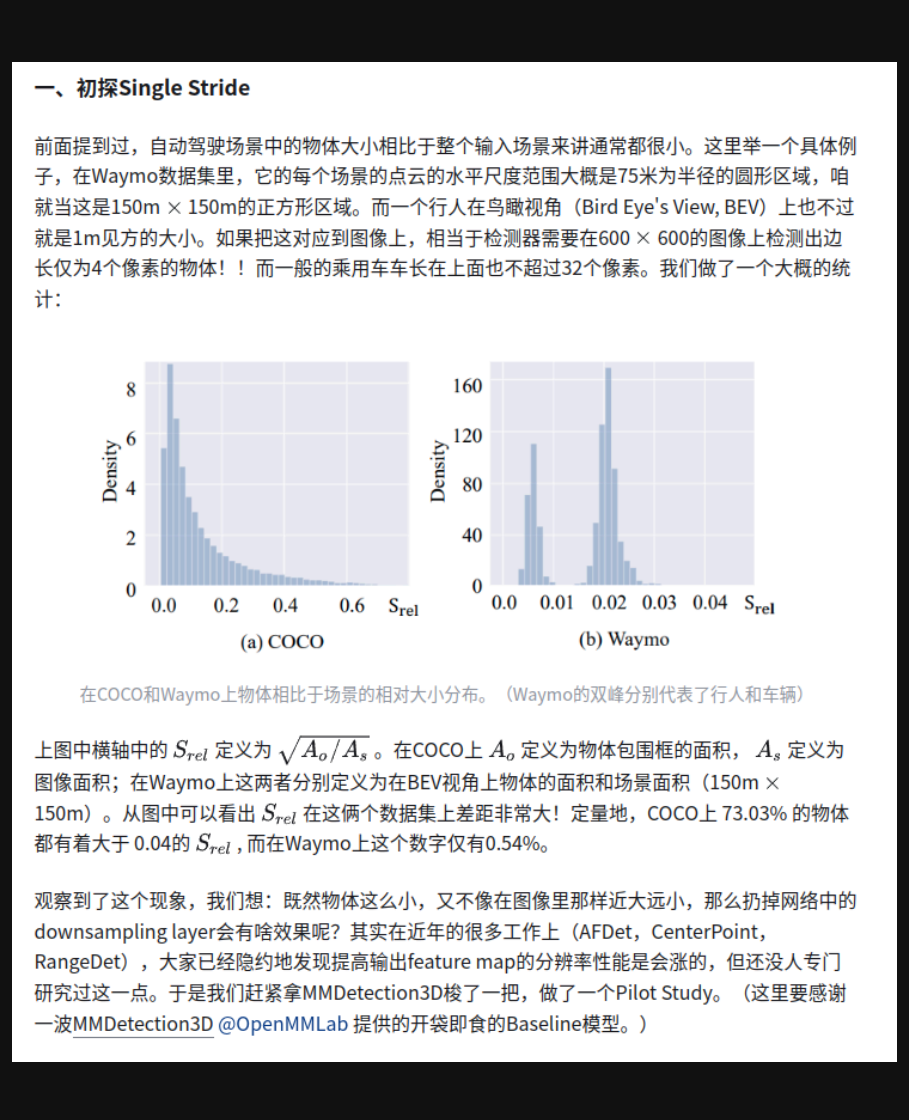
Dynamic Sparse Window Attent

Single Stride

Why point cloud images are sparse?

1. It only illuminates the surface of the obstacle and there is always occlusion
2. No scale variation. no near large or far small
3. scattering





一、初探Single Stride

前面提到过，自动驾驶场景中的物体大小相比于整个输入场景来讲通常都很小。这里举一个具体例子，在Waymo数据集里，它的每个场景的点云的水平尺度范围大概是75米为半径的圆形区域，咱就当这是150m × 150m的正方形区域。而一个行人在鸟瞰视角（Bird Eye's View, BEV）上也不过就是1m见方的大小。如果把这对应到图像上，相当于检测器需要在600 × 600的图像上检测出边长仅为4个像素的物体！！而一般的乘用车车长在上面也不超过32个像素。我们做了一个大概的统计：

在COCO和Waymo上物体相比于场景的相对大小分布。（Waymo的双峰分别代表了行人和车辆）

上图中横轴中的 S_{rel} 定义为 $\sqrt{A_o/A_s}$ 。在COCO上 A_o 定义为物体包围框的面积， A_s 定义为图像面积；在Waymo上这两者分别定义为在BEV视角上物体的面积和场景面积（150m × 150m）。从图中可以看出 S_{rel} 在这俩个数据集上差距非常大！定量地，COCO上 73.03% 的物体都有着大于 0.04的 S_{rel} ，而在Waymo上这个数字仅有0.54%。

观察到了这个现象，我们想：既然物体这么小，又不像在图像里那样近大远小，那么扔掉网络中的downsampling layer会有啥效果呢？其实在近年的很多工作上（AFDet，CenterPoint，RangeDet），大家已经隐约地发现提高输出feature map的分辨率性能是会涨的，但还没人专门研究过这一点。于是我们赶紧拿MMDetection3D梭了一把，做了一个Pilot Study。（这里要感谢一波MMDetection3D @OpenMMLab 提供的开袋即食的Baseline模型。）

Concrete

- Dynamic Sparse Window Attention
  a. "efficiently process sparse points in parallel"
- Rotated set partitioning strategy
  a. "allow the cross-set connection in consecutive self-attention layers"

Structure

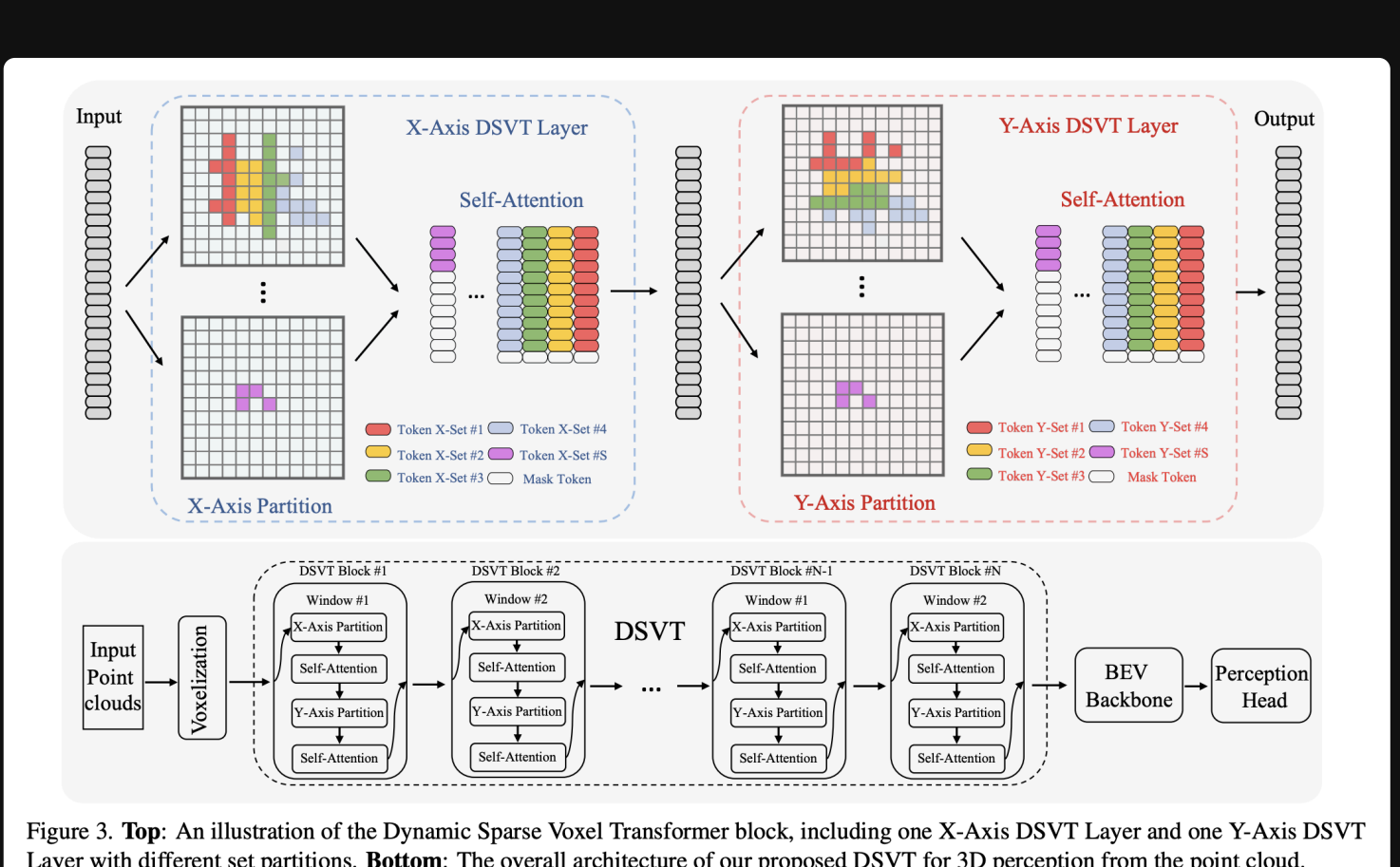


Figure 3. **Top**: An illustration of the Dynamic Sparse Voxel Transformer block, including one X-Axis DSVT Layer and one Y-Axis DSVT Layer with different set partitions. **Bottom**: The overall architecture of our proposed DSVT for 3D perception from the point cloud.

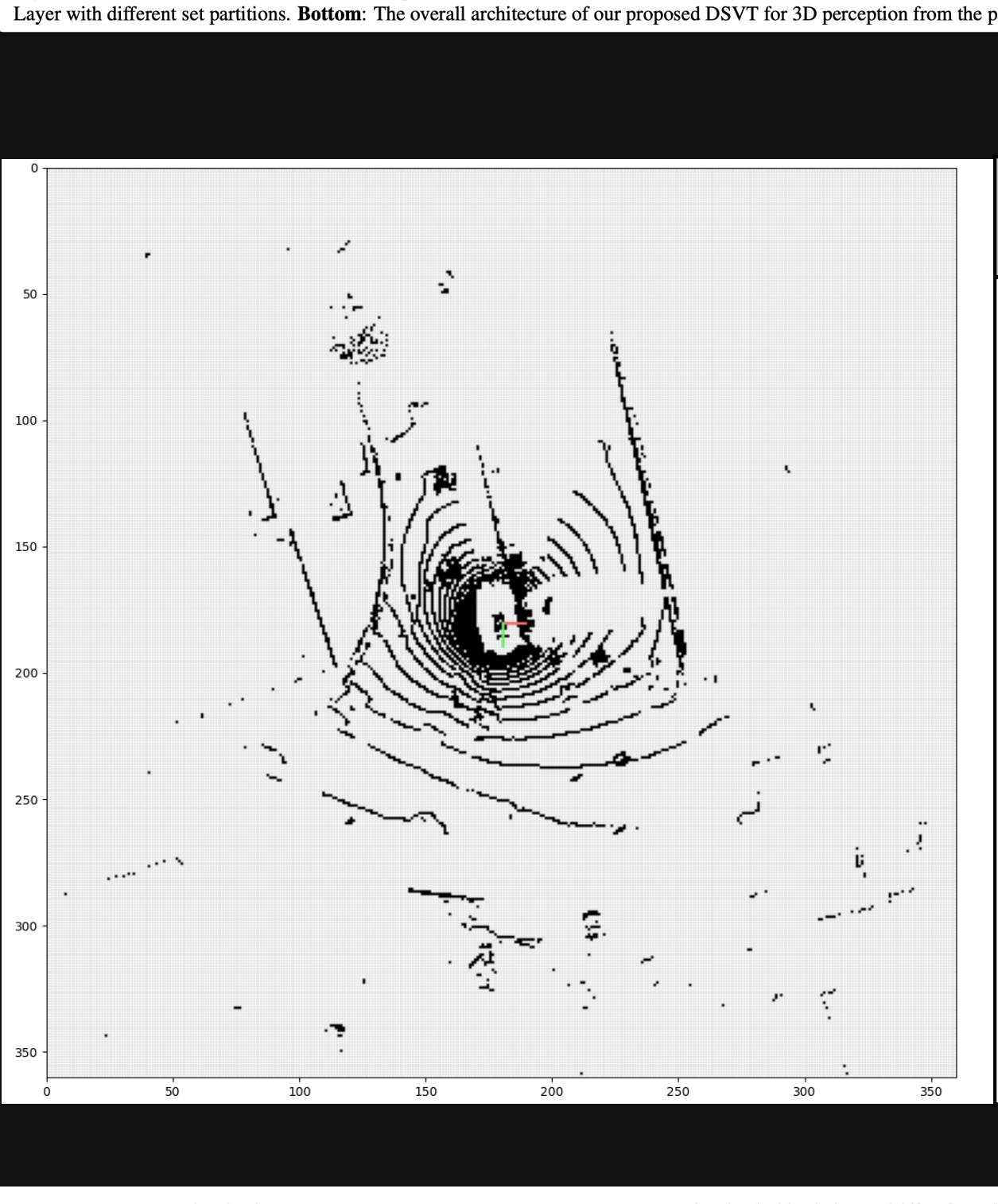
Input

Voxelization

3D Backbone

BEV Backbone

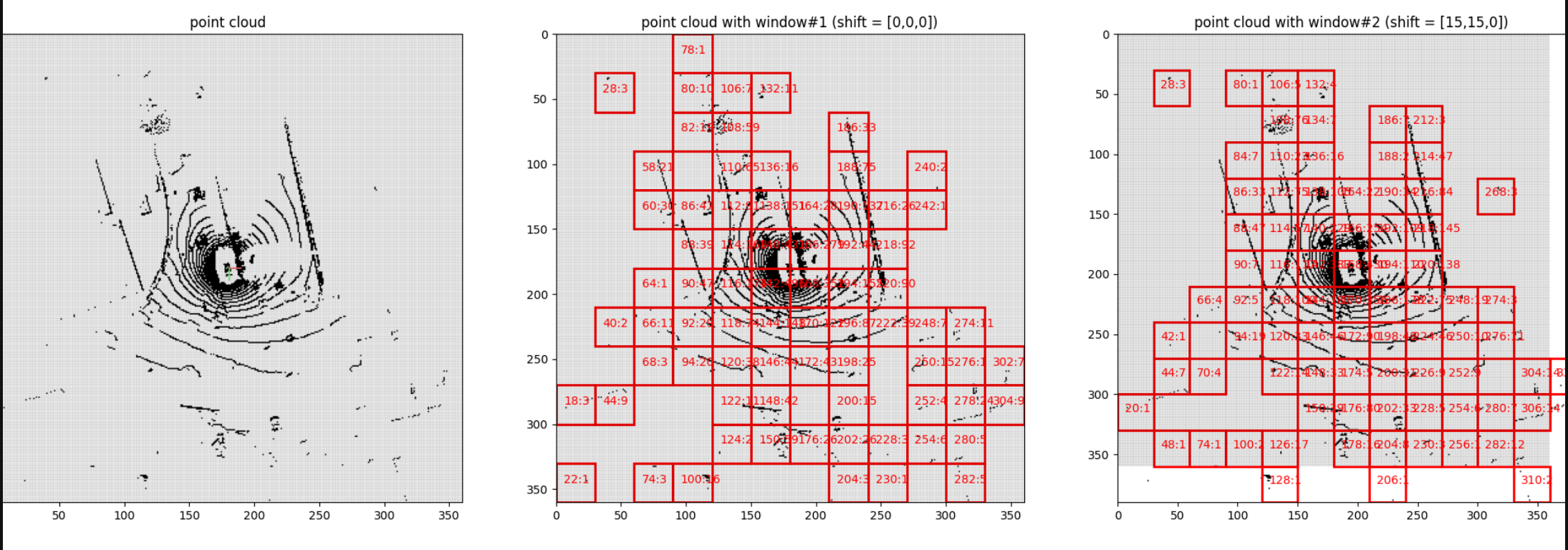
Head



1. Raw_cloud to Voxels

(n, 3[x, y, z]) >> (m, 4 [batch_id, z, y, z])

- pc_range = [-54, -54, -5, 54, 54, 3]
- voxel_size = [0.3, 0.3, 8]
- H = (54+54)/0.3 = 360
- W = (54+54)/0.3 = 360
- max_voxels = 360*360=129600
- num_voxels = 4200



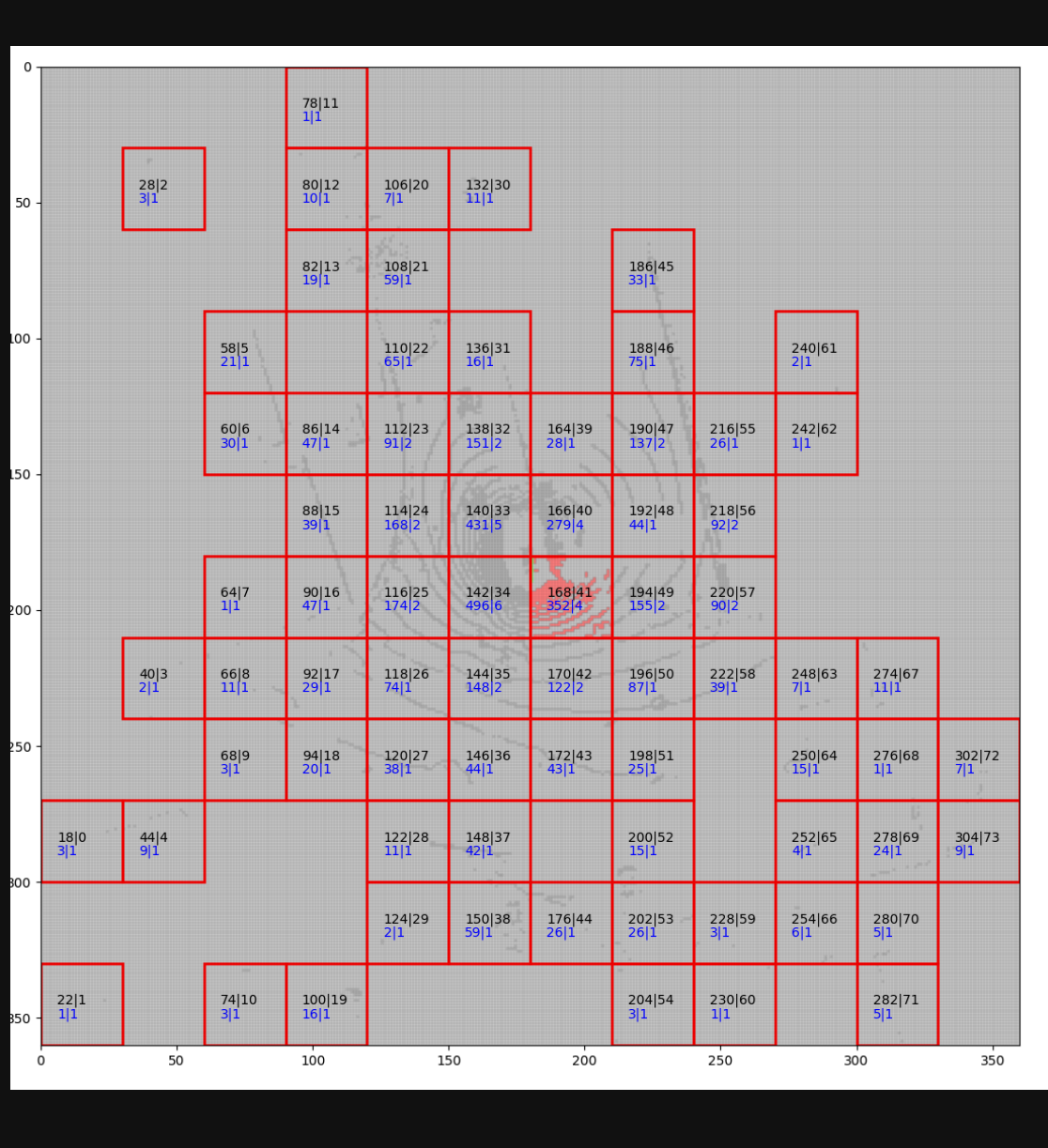
2. Voxels to Token

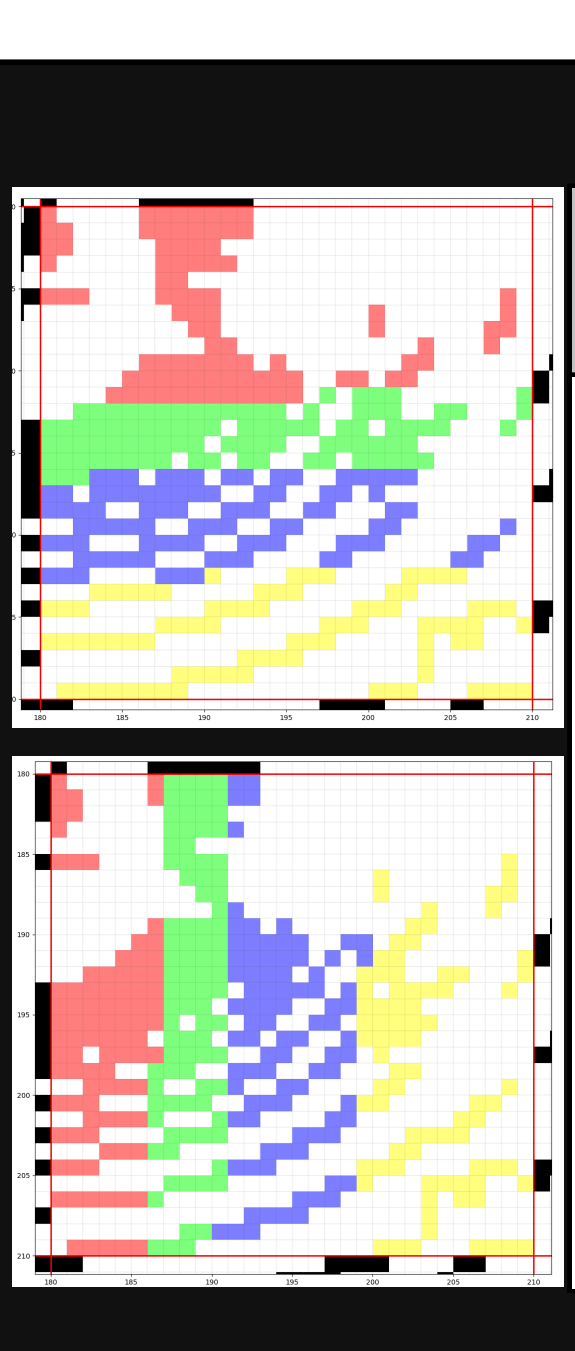
2.1 generate local set

- non-overlapped
- size-equivalent

2.1.1 window partition

- window_shape: [30, 30, 1]
- shift_list: [15, 15, 0]
- size_win_x: 360 / 30 = 12
- size_win_y: 360 / 30 = 12
- size_win_z: 1 / 1 = 1
- max_win_per_sample: 13*13*2 = 338





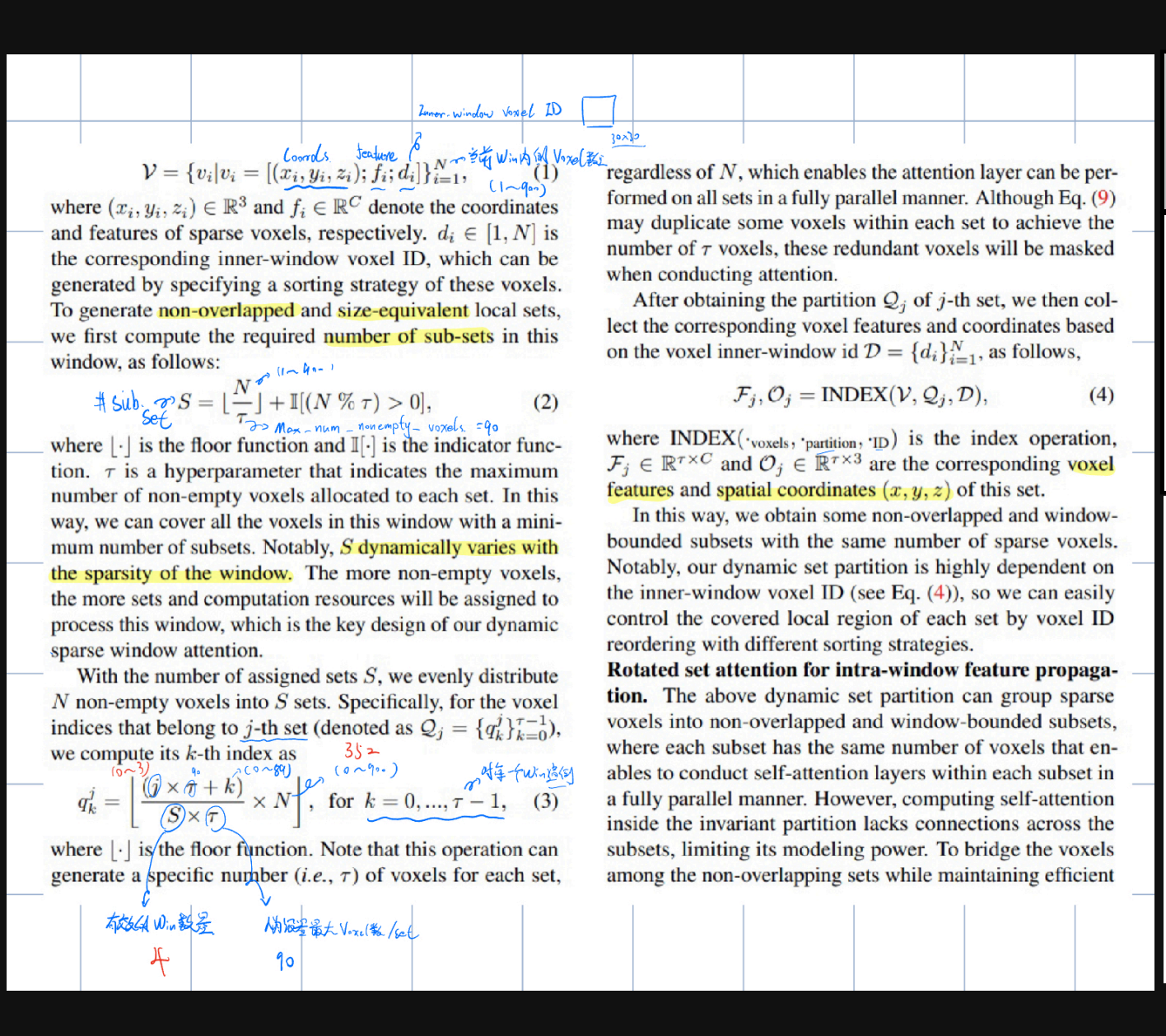
2. Voxels to Token

2.1 generate local set

- non-overlapped
- size-equivalent

2.1.2 get_set

- max_voxel_per_set = 90
- num_win = 74
- num_set = 98
- token_shape = (2, 98, 90)



$\mathcal{V} = \{v_i | v_i = [(x_i, y_i, z_i); f_i; d_i]\}_{i=1}^{N},$ (1)

where $(x_i, y_i, z_i) \in \mathbb{R}^3$ and $f_i \in \mathbb{R}^C$ denote the coordinates and features of sparse voxels, respectively. $d_i \in [1, N]$ is the corresponding inner-window voxel ID, which can be generated by specifying a sorting strategy of these voxels. To generate non-overlapped and size-equivalent local sets, we first compute the required number of sub-sets in this window, as follows:

$$S = \lfloor \frac{N}{\tau} \rfloor + \mathbb{I}[(N \,\%\, \tau) > 0], \quad (2)$$

where $\lfloor\cdot\rfloor$ is the floor function and $\mathbb{I}[\cdot]$ is the indicator function. τ is a hyperparameter that indicates the maximum number of non-empty voxels allocated to each set. In this way, we can cover all the voxels in this window with a minimum number of subsets. Notably, S dynamically varies with the sparsity of the window. The more non-empty voxels, the more sets and computation resources will be assigned to process this window, which is the key design of our dynamic sparse window attention.

With the number of assigned sets S, we evenly distribute N non-empty voxels into S sets. Specifically, for the voxel indices that belong to j-th set (denoted as $\mathcal{Q}_j = \{q_k^j\}_{k=0}^{\tau-1}$), we compute its k-th index as

$$q_k^j = \left\lfloor \frac{(j \times \tau + k)}{S \times \tau} \times N \right\rfloor, \text{ for } k = 0, ..., \tau - 1, \quad (3)$$

where $\lfloor\cdot\rfloor$ is the floor function. Note that this operation can generate a specific number (*i.e.*, τ) of voxels for each set, regardless of N, which enables the attention layer can be performed on all sets in a fully parallel manner. Although Eq. (9) may duplicate some voxels within each set to achieve the number of τ voxels, these redundant voxels will be masked when conducting attention.

After obtaining the partition $\mathcal{Q}_j$ of j-th set, we then collect the corresponding voxel features and coordinates based on the voxel inner-window id $\mathcal{D} = \{d_i\}_{i=1}^{N}$, as follows,

$$\mathcal{F}_j, \mathcal{O}_j = \text{INDEX}(\mathcal{V}, \mathcal{Q}_j, \mathcal{D}), \quad (4)$$

where INDEX($\cdot_{\text{voxels}}, \cdot_{\text{partition}}, \cdot_{\text{ID}}$) is the index operation, $\mathcal{F}_j \in \mathbb{R}^{\tau \times C}$ and $\mathcal{O}_j \in \mathbb{R}^{\tau \times 3}$ are the corresponding voxel features and spatial coordinates (x, y, z) of this set.

In this way, we obtain some non-overlapped and window-bounded subsets with the same number of sparse voxels. Notably, our dynamic set partition is highly dependent on the inner-window voxel ID (see Eq. (4)), so we can easily control the covered local region of each set by voxel ID reordering with different sorting strategies.

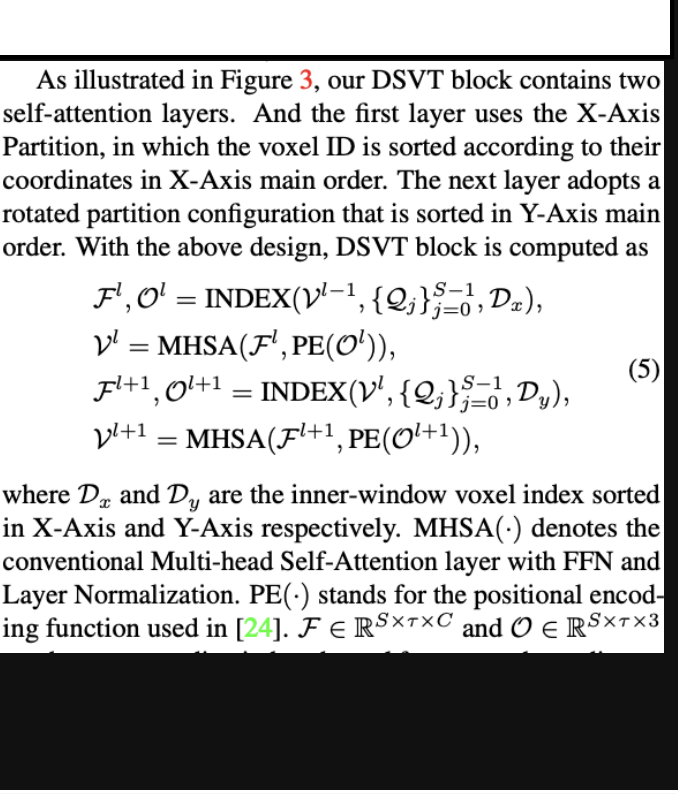
Rotated set attention for intra-window feature propagation. The above dynamic set partition can group sparse voxels into non-overlapped and window-bounded subsets, where each subset has the same number of voxels that enables to conduct self-attention layers within each subset in a fully parallel manner. However, computing self-attention inside the invariant partition lacks connections across the subsets, limiting its modeling power. To bridge the voxels among the non-overlapping sets while maintaining efficient

2. Voxels to Token

2.1 generate local set

- non-overlapped
- size-equivalent

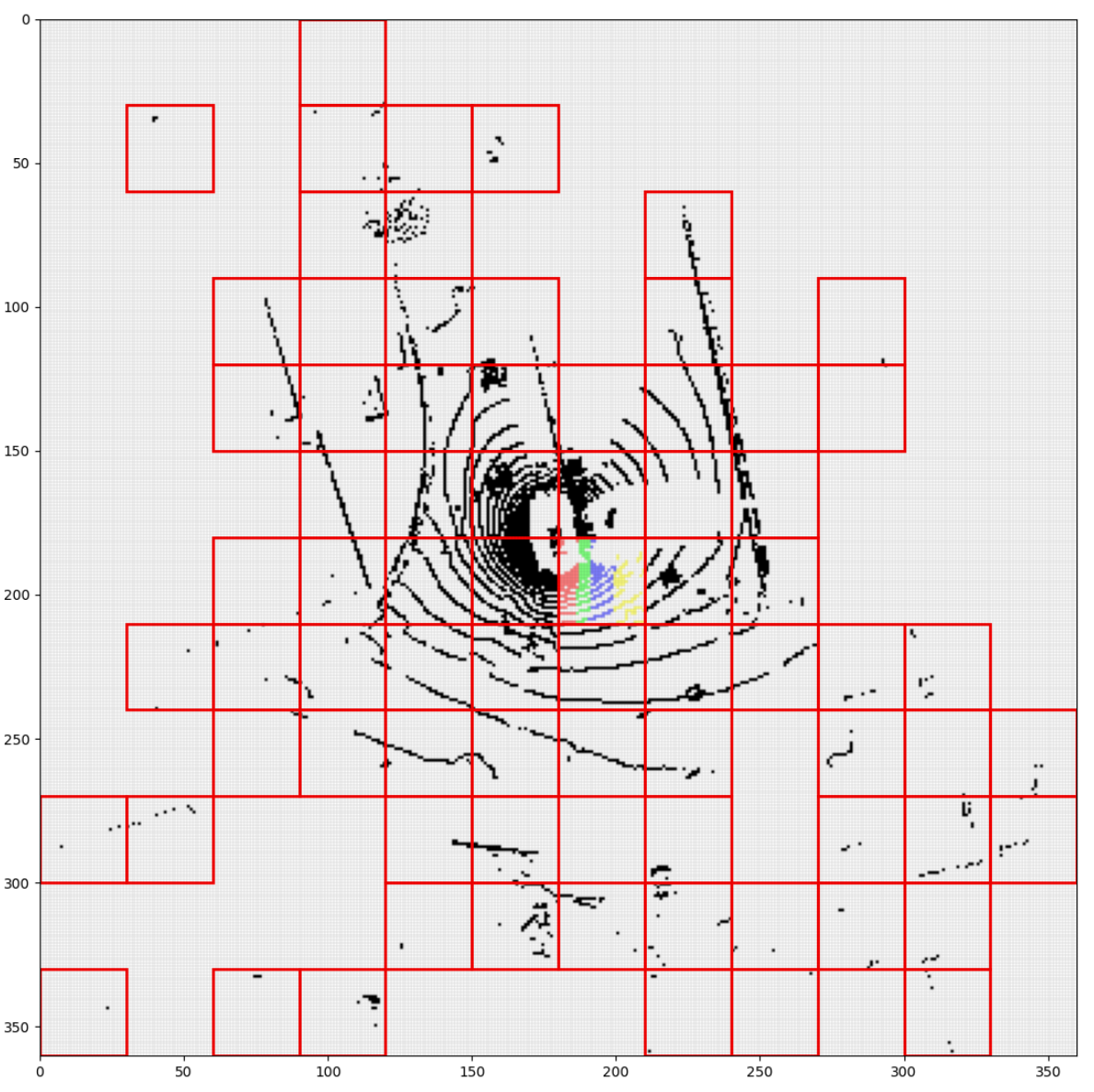
2.1.3 set to token

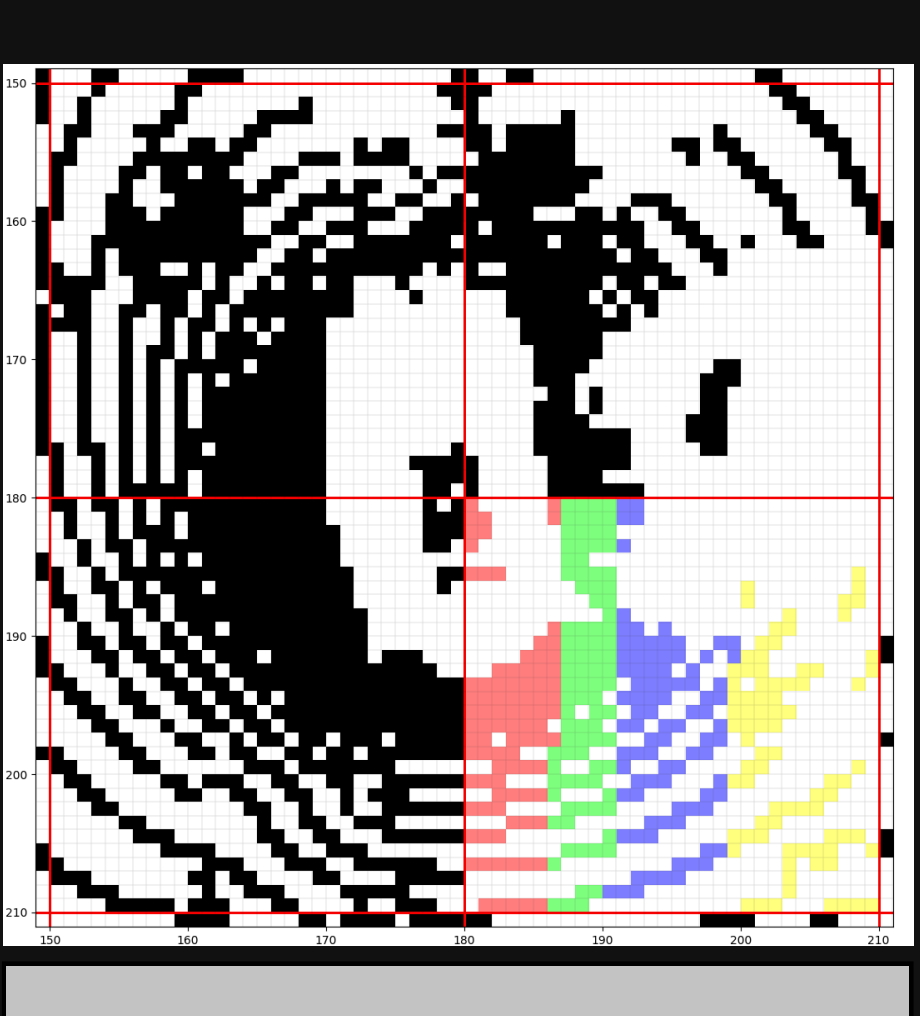


As illustrated in Figure 3, our DSVT block contains two self-attention layers. And the first layer uses the X-Axis Partition, in which the voxel ID is sorted according to their coordinates in X-Axis main order. The next layer adopts a rotated partition configuration that is sorted in Y-Axis main order. With the above design, DSVT block is computed as

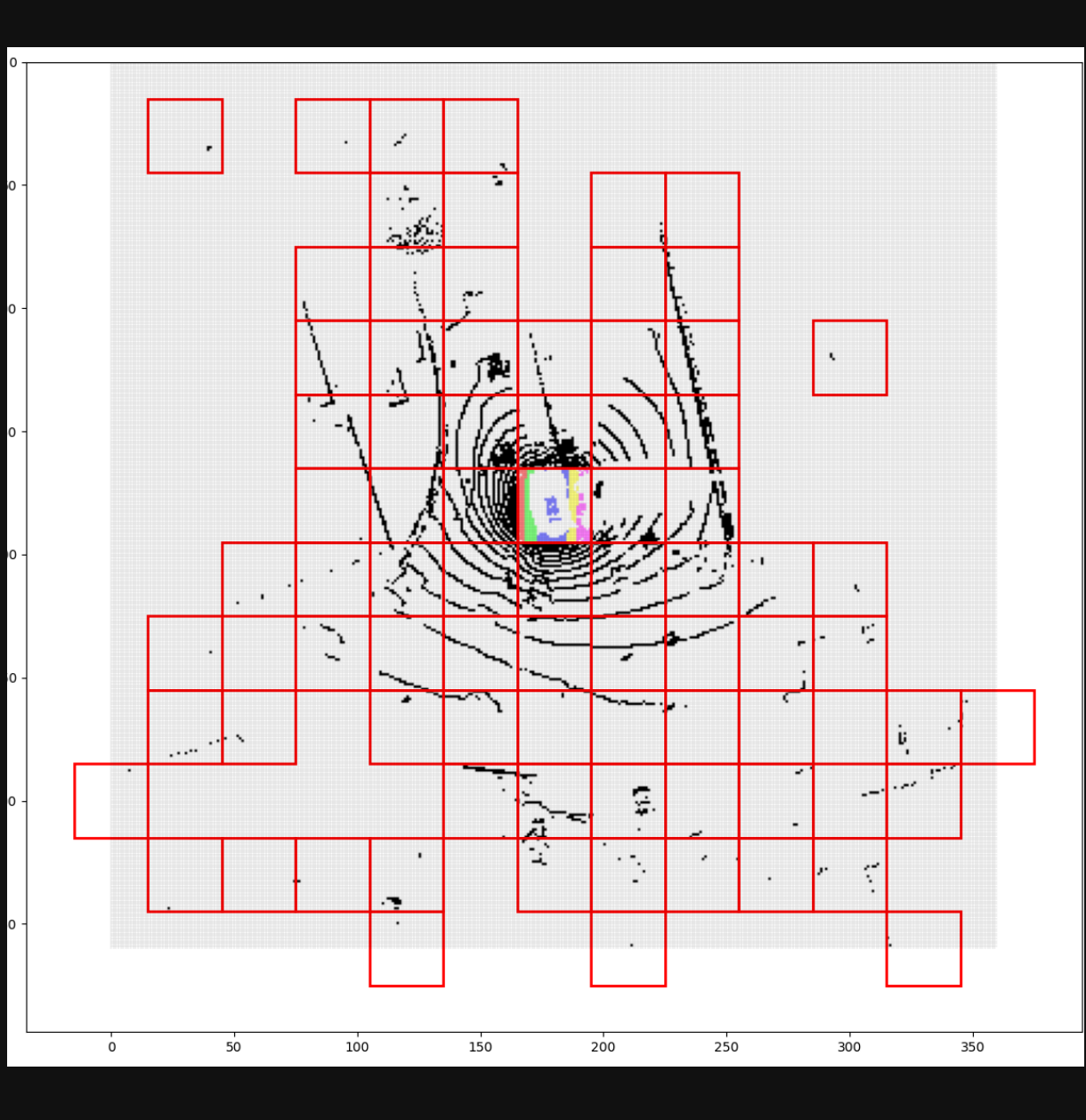
$$\begin{aligned}
\mathcal{F}^l, \mathcal{O}^l &= \text{INDEX}(\mathcal{V}^{l-1}, \{\mathcal{Q}_j\}_{j=0}^{S-1}, \mathcal{D}_x), \\
\mathcal{V}^l &= \text{MHSA}(\mathcal{F}^l, \text{PE}(\mathcal{O}^l)), \\
\mathcal{F}^{l+1}, \mathcal{O}^{l+1} &= \text{INDEX}(\mathcal{V}^l, \{\mathcal{Q}_j\}_{j=0}^{S-1}, \mathcal{D}_y), \\
\mathcal{V}^{l+1} &= \text{MHSA}(\mathcal{F}^{l+1}, \text{PE}(\mathcal{O}^{l+1})),
\end{aligned} \quad (5)$$

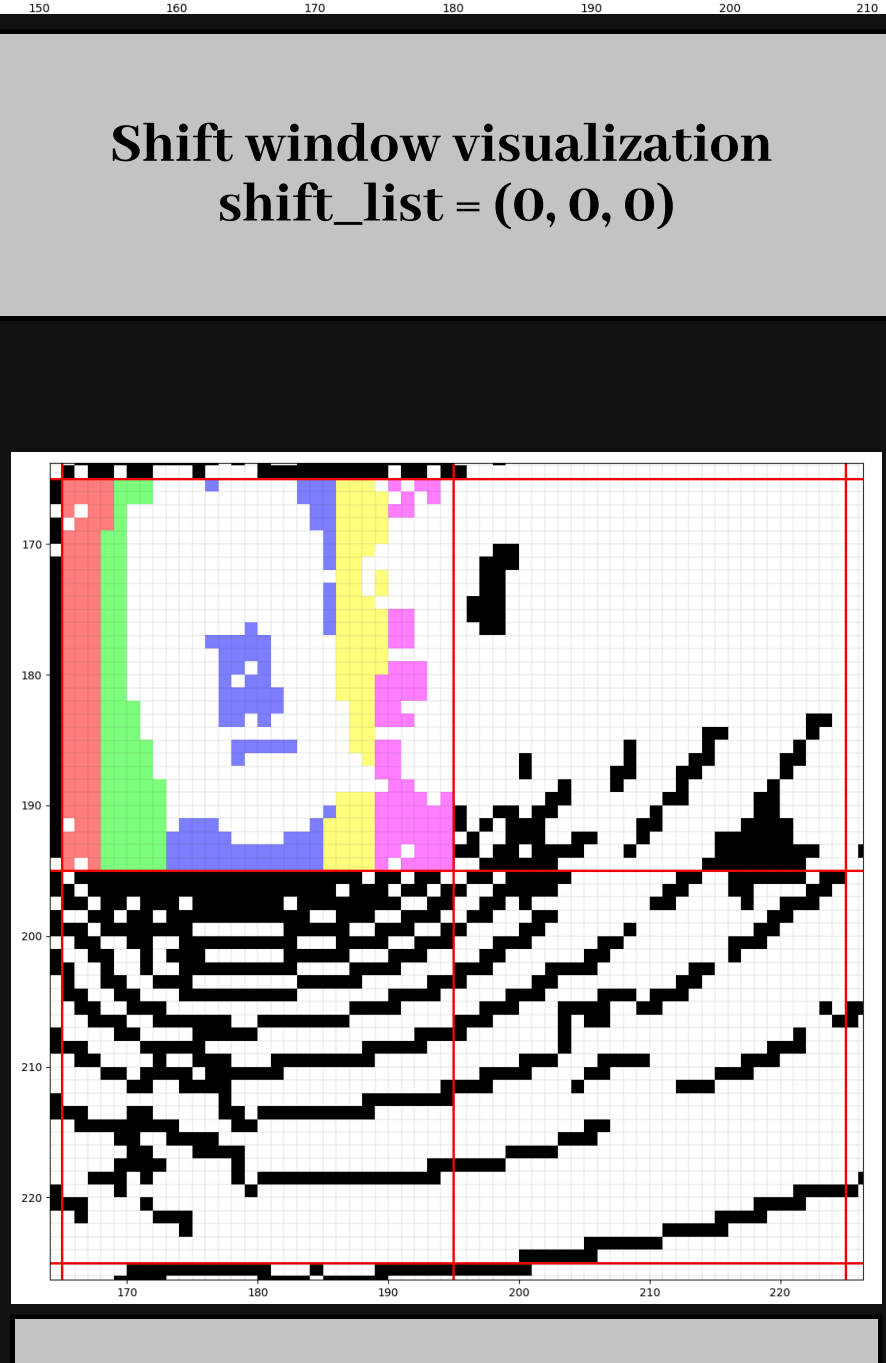
where $\mathcal{D}_x$ and $\mathcal{D}_y$ are the inner-window voxel index sorted in X-Axis and Y-Axis respectively. MHSA$(\cdot)$ denotes the conventional Multi-head Self-Attention layer with FFN and Layer Normalization. PE$(\cdot)$ stands for the positional encoding function used in [24]. $\mathcal{F} \in \mathbb{R}^{S \times \tau \times C}$ and $\mathcal{O} \in \mathbb{R}^{S \times \tau \times 3}$



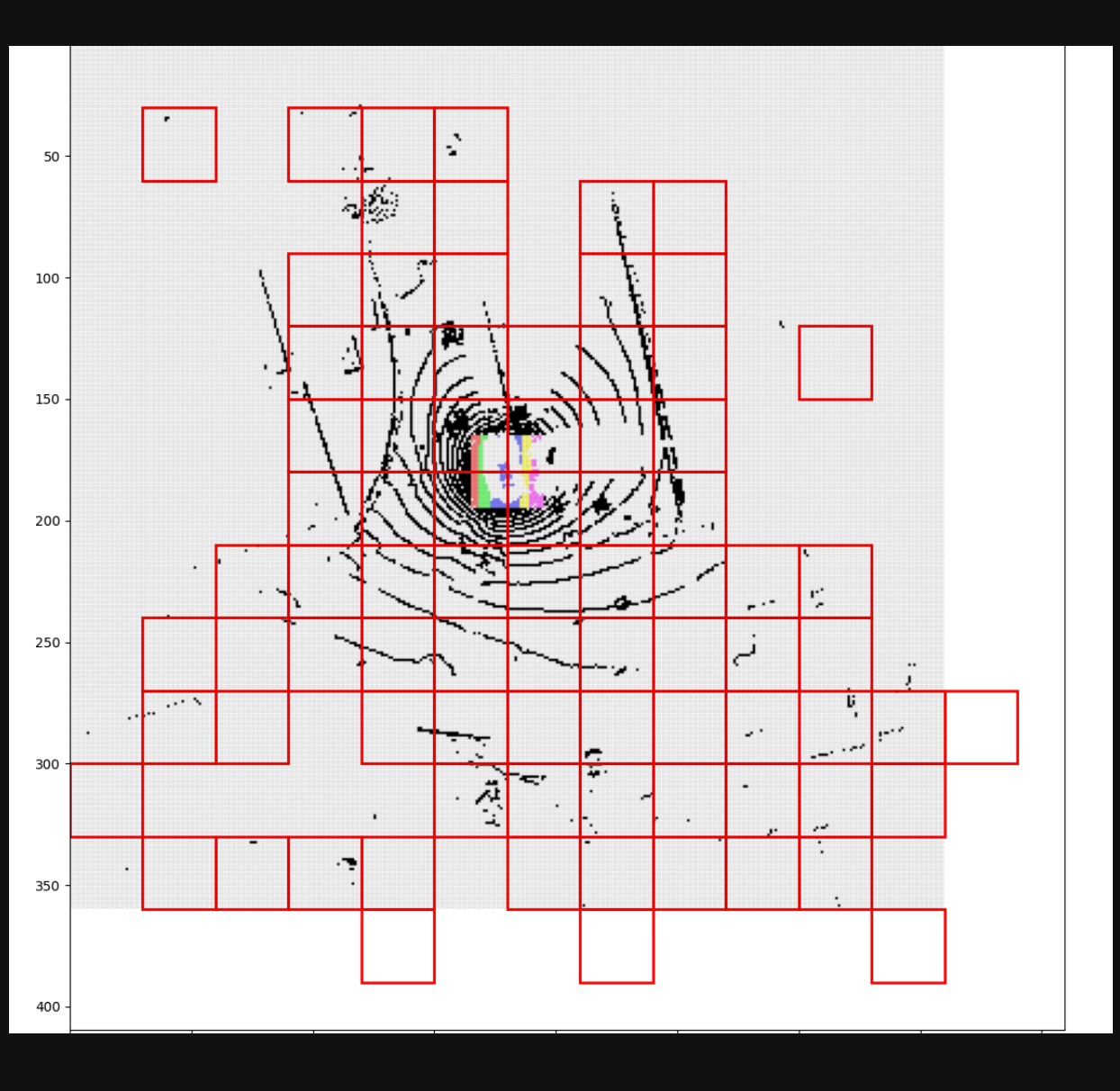


Shift window visualization
shift_list = (0, 0, 0)





Shift window visualization
shift_list = (15, 15, 0)



Shift Window